

Nonlinear manifold approximation using Compositional Polynomial Networks

Joël Soffo

Airbus

Centrale Nantes, Laboratoire de Mathématiques Jean Leray

February 22, 2026

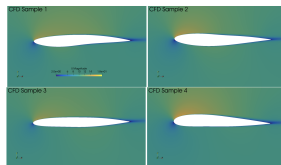
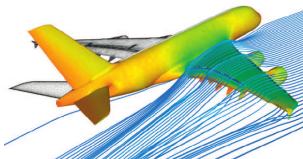


AIRBUS

1 - Introduction

- 1 Introduction
- 2 Linear approximation
- 3 Nonlinear approximation
- 4 Decoder based on compositional polynomial networks
- 5 Numerical illustrations

- Many (industrial-scale) problems are time and resource intensive,
- Parametric problems in particular are challenging,
 - ▶ solving the High Dimensional Model (HDM) for each value of the parameter is expensive.
- Reduced-order models can be used as surrogate,
 - ▶ The speed-up compared to the HDM is non-negligible.



We want to find the solution of a given system,

Differential equation

$$\Psi_{[\mu]}u = f.$$

- u is the solution,
- μ represents the parameters,
- f is a source term.

$u \in \mathbb{R}^{\mathcal{N}}, f \in \mathbb{R}^{\mathcal{N}}$ with $\mathcal{N} \gg 1$ (discretized functions).

We want to find the solution of a given system,

Differential equation

$$\Psi_{[\mu]}u = f.$$

- u is the solution,
- μ represents the parameters,
- f is a source term.

$u \in \mathbb{R}^{\mathcal{N}}, f \in \mathbb{R}^{\mathcal{N}}$ with $\mathcal{N} \gg 1$ (discretized functions).

~> The system can be computationally expensive to solve .

Reduced order modeling

$$(\mu, f) \xrightarrow{F} a \begin{array}{c} \xrightarrow{D} u \\ \xleftarrow{E} \end{array}$$

- $a \in \mathbb{R}^n$, $\tilde{u} \approx u \in \mathbb{R}^{\mathcal{N}}$, $n \ll \mathcal{N}$.
- **Offline phase** : We have access to samples $\{u_i\}_{i=1}^{N_d}$, we construct E and D .

$$(\mu, f) \xrightarrow{F} \boxed{a \begin{array}{c} \xrightarrow{D} u \\ \xleftarrow{E} \end{array}}$$

- **Online phase** : For a new value (μ, f) , we only have to compute F (fast to compute).

$$\boxed{(\mu, f) \xrightarrow{F} a} \begin{array}{c} \xrightarrow{D} u \\ \xleftarrow{E} \end{array}$$

$\leadsto$ We focus here on the offline phase.

Reduced order models

$\rightsquigarrow$ How to go from u to a , and from a to u ?

Encoder :

$$E : \mathbb{R}^{\mathcal{K}} \longrightarrow \mathbb{R}^n$$

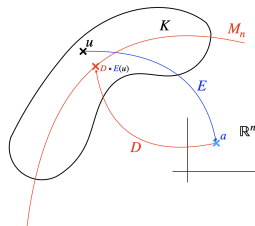
$$u \longmapsto a$$

Decoder :

$$D : \mathbb{R}^n \longrightarrow \mathbb{R}^{\mathcal{K}}$$

$$a \longmapsto \tilde{u}$$

$$K = \{u(\mu), \mu \in \mathbb{R}^p\}, M_n = \{D(a), a \in \mathbb{R}^n\}.$$



Reduced order models

↪ How to go from u to a , and from a to u ?

Encoder :

$$E : \mathbb{R}^{\mathcal{K}} \longrightarrow \mathbb{R}^n$$

$$u \longmapsto a$$

Decoder :

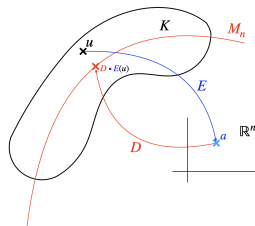
$$D : \mathbb{R}^n \longrightarrow \mathbb{R}^{\mathcal{K}}$$

$$a \longmapsto \tilde{u}$$

$$K = \{u(\mu), \mu \in \mathbb{R}^p\}, M_n = \{D(a), a \in \mathbb{R}^n\}.$$

↪ We want $u \approx D \circ E(u)$ (for a given metric).

↪ The smaller n , the better.



2 - Linear approximation

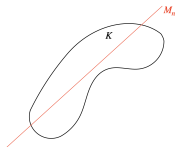
- 1 Introduction
- 2 Linear approximation**
- 3 Nonlinear approximation
- 4 Decoder based on compositional polynomial networks
- 5 Numerical illustrations

Linear approximation

A first idea is to choose the decoder D to be a **linear function** (linear approximation methods).

$$\begin{aligned} D \circ E(u) &= D(E(u)) \\ &= D(a) \\ &= Va \end{aligned}$$

with $a = E(u) \in \mathbb{R}^n, V \in \mathbb{R}^{\mathcal{N} \times n}$.

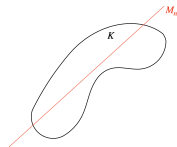


Linear approximation

A first idea is to choose the decoder D to be a **linear function** (linear approximation methods).

$$\begin{aligned} D \circ E(u) &= D(E(u)) \\ &= D(a) \\ &= Va \end{aligned}$$

with $a = E(u) \in \mathbb{R}^n, V \in \mathbb{R}^{\mathcal{X} \times n}$.



Disadvantage

For several problems, linear methods are limited. We need a high n to have $u \approx Va$.

3 - Nonlinear approximation

- 1 Introduction
- 2 Linear approximation
- 3 Nonlinear approximation**
- 4 Decoder based on compositional polynomial networks
- 5 Numerical illustrations

Nonlinear approximation

- Nonlinear methods consists in taking the decoder D as **a nonlinear function**.
- A nonlinear decoder can lead to $u \approx D \circ E(u)$ with n **smaller** compared to linear methods.

Famous example :

Auto-encoders belongs to the category of nonlinear methods. The encoder and the decoder are neural networks.

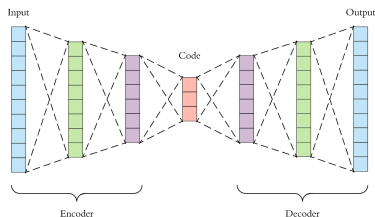


Figure: Auto-encoder architecture ¹

¹Arden Dertat : Applied Deep Learning - Part 3: Autoencoders

Linear encoder and nonlinear decoder

Consider a basis $\phi_1, \dots, \phi_N$ of a linear space X_N .

Suppose that we have $u \approx D(a) = \sum_{i=1}^N a_i \phi_i$ (N can be *large*).

We went from $u \in \mathbb{R}^{\mathcal{N}}$ to $a \in \mathbb{R}^N$ (Not enough if N is *large*).

Linear encoder and nonlinear decoder

Consider a basis $\varphi_1, \dots, \varphi_N$ of a linear space X_N .

Suppose that we have $u \approx D(a) = \sum_{i=1}^N a_i \varphi_i$ (N can be *large*).

We went from $u \in \mathbb{R}^{\mathcal{N}}$ to $a \in \mathbb{R}^N$ (Not enough if N is *large*).

Idea :

Write the decoder in the form

$$D(a) = \sum_{i=1}^n a_i \varphi_i + \sum_{i=n+1}^N g_i(a) \varphi_i$$

where the functions $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ are **nonlinear maps**, $a = (a_1, \dots, a_n)$.

Linear encoder and nonlinear decoder

Consider a basis $\varphi_1, \dots, \varphi_N$ of a linear space X_N .

Suppose that we have $u \approx D(a) = \sum_{i=1}^N a_i \varphi_i$ (N can be *large*).

We went from $u \in \mathbb{R}^{\mathcal{N}}$ to $a \in \mathbb{R}^N$ (Not enough if N is *large*).

Idea :

Write the decoder in the form

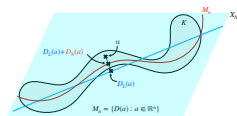
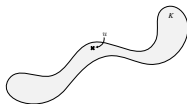
$$D(a) = \sum_{i=1}^n a_i \varphi_i + \sum_{i=n+1}^N g_i(a) \varphi_i$$

where the functions $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ are **nonlinear maps**, $a = (a_1, \dots, a_n)$.

$\rightsquigarrow$ We went from $u \in \mathbb{R}^{\mathcal{N}}$ to $a \in \mathbb{R}^n$.

Linear encoder and nonlinear decoder

$$D(a) = \underbrace{\sum_{i=1}^n a_i \varphi_i}_{D_L(a)} + \underbrace{\sum_{i=n+1}^N g_i(a) \varphi_i}_{D_N(a)}$$



The structure of the decoder relies on the fact that for

$$u = \sum_{i=1}^n a_i \varphi_i + \sum_{i>n} a_i \varphi_i$$

the coefficients $a_i(u)$ for $i > n$ may be well approximated as functions $g_i(E(u))$ of the first few coefficients $E(u) = (a_i(u))_{i=1}^n$.

The structure of the decoder relies on the fact that for

$$u = \sum_{i=1}^n a_i \varphi_i + \sum_{i>n} a_i \varphi_i$$

the coefficients $a_i(u)$ for $i > n$ may be well approximated as functions $g_i(E(u))$ of the first few coefficients $E(u) = (a_i(u))_{i=1}^n$.

State of the art methods include

- Quadratic polynomials [Barnett and Farhat 2022][Geelen et al 2023]
- Sums of univariate high-order polynomials [Geelen et al 2024]
- Neural networks or random forests [Barnett, Farhat and Maday 2023], [Cohen, Farhat, Maday, Somacal 2023]

Quadratic approximations

- $\phi_1, \dots, \phi_n$ are orthonormal vectors chosen based on empirical PCA.

$$E(u) = (a_i(u))_{i=1}^n = ((u|\phi_i))_{i=1}^n.$$

$$D(a) = \sum_{i=1}^n a_i \phi_i + \sum_{1 \leq i < j \leq n} a_i a_j \phi_{i,j}.$$

- The vectors $\phi_{i,j}, 1 \leq i < j \leq n$ are solutions of the optimization problem

$$\min_{\phi_{i,j}} \sum_{k=1}^m \|u_k - D(E(u_k))\|_X^2.$$

- $\phi_1, \dots, \phi_n$ are orthonormal vectors chosen based on empirical PCA.

$$E(u) = (a_i(u))_{i=1}^n = ((u|\phi_i))_{i=1}^n.$$

$$D(a) = \sum_{i=1}^n a_i \phi_i + \sum_{i=n+1}^N g_i(a) \phi_i$$

- $g : \mathbb{R}^n \rightarrow \mathbb{R}^{N-n}$ is a neural network trained using data $\{a^k, b^k\}_{k=1}^m$ where $a^k = (a_1^k, \dots, a_n^k)^T \in \mathbb{R}^n$ and $b^k = (a_{n+1}^k, \dots, a_N^k) \in \mathbb{R}^{N-n}$.

- There is no rule to choose n and N .
- Taking the first n components for the linear part is not optimal (at least for quadratic approximations, [Schwerdtner, Peherstorfer 2024]).
- The relation between $a_i(u)$, $i > n$ and $E(u)$ may be highly nonlinear. Even highly expressive approximation tools may result in poor accuracy, due to the difficulty of learning with limited data.

4 - Decoder based on compositional polynomial networks

- 1 Introduction
- 2 Linear approximation
- 3 Nonlinear approximation
- 4 Decoder based on compositional polynomial networks**
- 5 Numerical illustrations

Decoder based on compositional polynomial network

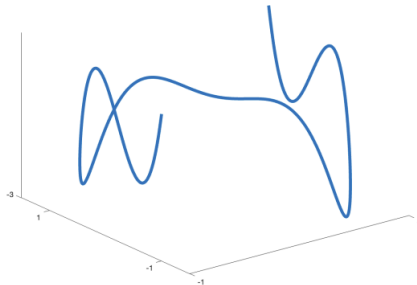
~> We propose a new parameterization for the functions g_i .

In many applications, a coefficient $a_i(u)$ for $i > n$ may have a highly nonlinear relation with the first n coefficients $a = E(u)$ but a much smoother relation when expressed in terms of a and additional coefficients $a_j(u)$ with $n < j < i$.

Decoder based on compositional polynomial network

→ We propose a new parameterization for the functions g_i .

In many applications, a coefficient $a_i(u)$ for $i > n$ may have a **highly nonlinear relation** with the first n coefficients $a = E(u)$ but a **much smoother relation** when expressed in terms of a and additional coefficients $a_j(u)$ with $n < j < i$.

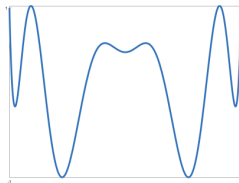


(a) $K = \{(a_1(t), a_2(t), a_3(t)) : t \in [0, 1]\}$

Decoder based on compositional polynomial network



(b) a_2 as function of a_1



(c) a_3 as function of a_1

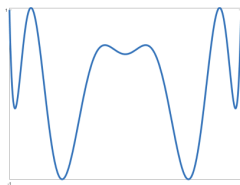
For $u = (a_1, a_2, a_3)$,

instead of $u \approx (a_1, g_2(a_1), g_3(a_1))$,

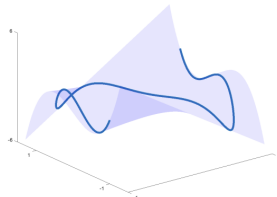
Decoder based on compositional polynomial network



(b) a_2 as function of a_1



(c) a_3 as function of a_1



(d) a_3 as function of (a_1, a_2)

For $u = (a_1, a_2, a_3)$,

instead of $u \approx (a_1, g_2(a_1), g_3(a_1))$,

we do $u \approx (a_1, g_2(a_1), f_3(a_1, g_2(a_1)))$

Decoder based on compositional polynomial network

This suggests the following compositional structure of the decoder's functions

$$g_i(a) = f_i(a, (g_j(a))_{n < j \leq n_i}),$$

where the f_i are polynomial functions.

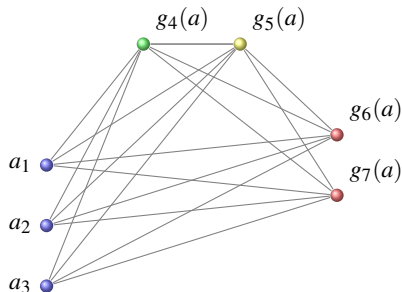


Figure: A compositional polynomial network (CPN) with $N = 7$ and $n = 3$.

Decoder based on compositional polynomial network

More generally, we will consider

- A set I ,
- its complementary set I^c such that $I \cup I^c = \{1, \dots, N\}$.

$$D(a) = \sum_{i=1}^N g_i(a) \varphi_i \quad \left(= \sum_{i \in I} a_i \varphi_i + \sum_{i \in I^c} g_i(a) \varphi_i \right)$$

where

- $g_i(a) = a_i$ if $i \in I$,
- $g_i(a) = f_i(g_1(a), \dots, g_{n_i}(a))$, with $n_i < i$ if $i \in I^c$,
- $f_i : \mathbb{R}^{n_i} \rightarrow \mathbb{R}$ is a n_i -variate polynomial.

Decoder based on compositional polynomial network

More generally, we will consider

- A set I ,
- its complementary set I^c such that $I \cup I^c = \{1, \dots, N\}$.

$$D(a) = \sum_{i=1}^N g_i(a) \varphi_i \quad \left(= \sum_{i \in I} a_i \varphi_i + \sum_{i \in I^c} g_i(a) \varphi_i \right)$$

where

- $g_i(a) = a_i$ if $i \in I$,
- $g_i(a) = f_i(g_1(a), \dots, g_{n_i}(a))$, with $n_i < i$ if $i \in I^c$,
- $f_i : \mathbb{R}^{n_i} \rightarrow \mathbb{R}$ is a n_i -variate polynomial.

I, I^c and n_i are obtained using an adaptive strategy.

Control of error (mean-squared setting)

The mean squared error

$$e_2(D \circ E) := \|id - D \circ E\|_2 := \left(\int_K \|u - D(E(u))\|_X^2 d\rho(u) \right)^{1/2}$$

$$e_2(D \circ E)^2 = \sum_{i \in I^c} \varepsilon_{i,2}^2 + \|id - P_{X_N}\|_2^2, \quad \varepsilon_{i,2} = \|a_i - g_i \circ a\|_2$$

Control of error (mean-squared setting)

The mean squared error

$$e_2(D \circ E) := \|id - D \circ E\|_2 := \left(\int_K \|u - D(E(u))\|_X^2 d\rho(u) \right)^{1/2}$$

$$e_2(D \circ E)^2 = \sum_{i \in I^c} \varepsilon_{i,2}^2 + \|id - P_{X_N}\|_2^2, \quad \varepsilon_{i,2} = \|a_i - g_i \circ a\|_2$$

Given a **prescribed precision** $0 < \varepsilon < 1$, it holds

$$e_2(D \circ E) \leq \varepsilon e_2(0)$$

whenever

$$\|id - P_{X_N}\|_2 \leq \beta \varepsilon e_2(0), \quad 0 < \beta < 1, \quad (1)$$

$$\varepsilon_{i,2}^2 \leq \tilde{\varepsilon}_{i,2}^2 := \omega_i (1 - \beta^2) \varepsilon^2 e_2(0)^2, \quad \forall i > n \quad (2)$$

where $(\omega_i)_{i \in I^c}$ are such that $\sum_{i \in I^c} \omega_i = 1$.

Control of error (mean-squared setting)

The mean squared error

$$e_2(D \circ E) := \|id - D \circ E\|_2 := \left(\int_K \|u - D(E(u))\|_X^2 d\rho(u) \right)^{1/2}$$

$$e_2(D \circ E)^2 = \sum_{i \in I^c} \varepsilon_{i,2}^2 + \|id - P_{X_N}\|_2^2, \quad \varepsilon_{i,2} = \|a_i - g_i \circ a\|_2$$

Given a **prescribed precision** $0 < \varepsilon < 1$, it holds

$$e_2(D \circ E) \leq \varepsilon e_2(0)$$

whenever

$$\|id - P_{X_N}\|_2 \leq \beta \varepsilon e_2(0), \quad 0 < \beta < 1, \quad (1)$$

$$\varepsilon_{i,2}^2 \leq \bar{\varepsilon}_{i,2}^2 := \omega_i (1 - \beta^2) \varepsilon^2 e_2(0)^2, \quad \forall i > n \quad (2)$$

where $(\omega_i)_{i \in I^c}$ are such that $\sum_{i \in I^c} \omega_i = 1$.

(1) is achieved by using PCA to define X_N , with a suitable selection of $N = N(\varepsilon)$.

(2) is achieved by a control of the approximation of a_i by $f_i((g_j(a))_{j=1}^{n_i})$.

Given an orthonormal basis $\varphi_1, \dots, \varphi_N$, the encoder $E : X \rightarrow \mathbb{R}^n$ is 1-Lipschitz

$$\boxed{\|E(u) - E(u')\|_2 = \|P_{X_n}(u - u')\|_X \leq \|u - u'\|_X} \quad \left(X_n := \text{span}\{\varphi_i : i \in I\}\right).$$

Control of stability

Given an orthonormal basis $\phi_1, \dots, \phi_N$, the encoder $E : X \rightarrow \mathbb{R}^n$ is 1-Lipschitz

$$\boxed{\|E(u) - E(u')\|_2 = \|P_{X_n}(u - u')\|_X \leq \|u - u'\|_X} \quad \left(X_n := \text{span}\{\phi_i : i \in I\} \right).$$

Given $\gamma = (\gamma_i)_{i=n+1}^N$, we equip $\mathbb{R}^{n_i}$ with the norm

$$\|b\|_{i,\gamma} = \max\{\|(b_j)_{j=1}^n\|_2, \max_{n < j \leq n_i} \gamma_j^{-1} |b_j|\}$$

and define the corresponding Lipschitz norm (estimated from samples)

$$\|f_i\|_{i,\gamma} = \max_{b,b'} \frac{|f_i(b) - f_i(b')|}{\|b - b'\|_{i,\gamma}}$$

Given a **prescribed Lipschitz constant** $L \geq 1$, letting $\|f_i\|_{i,\gamma} = \gamma_i$ and assuming $\gamma_i^2 \leq \tilde{\gamma}_i^2 := \omega_i(L^2 - 1)$ with $\sum_{i \in I^c} \omega_i \leq 1$, it holds

$$\boxed{\|D(a) - D(a')\|_X \leq L \|a - a'\|_2}$$

Polynomial approximation

↪ We set $d := n_i$.

↪ We have access to samples $x^{(k)} = (g_j(a(u^{(k)})))_{j=1}^d$ and $y^{(k)} = a_i(u^{(k)})$.

The function f_i is estimated from samples $\{x^{(k)}, y^{(k)}\}_{k=1}^m$, by solving the least-squares problem

$$\min_{f_i \in \mathcal{P}} \sum_{k=1}^m |f_i(x^{(k)}) - y^{(k)}|^2$$

where $\mathcal{P}$ is a set of polynomials over $\mathbb{R}^d$.

Sparse approximations (CPN-S)

$$\mathcal{P} = \mathbb{P}_\Lambda := \text{span}\{x^\lambda := x_1^{\lambda_1} \dots x_d^{\lambda_d} : \lambda \in \Lambda\},$$

Hyperbolic cross set with degree p : $\Lambda = \{\lambda \in \mathbb{N}^d : \prod_{j=1}^d (\lambda_j + 1) \leq p + 1\}$

Adaptive algorithm (glimpse !)

Inputs : Target precision ϵ , Lip. constant L , snapshot matrix $S \in \mathbb{R}^{\mathcal{N} \times m} (m < \mathcal{N})$.

$\rightsquigarrow$ Apply SVD on S to compute singular vectors $\phi_1, \phi_2, \dots, \phi_m$.

$\rightsquigarrow$ Find N such that $\|id - P_{X_N}\|_2 \leq \beta \epsilon e_2(0)$ with $X_N = \text{span}\{\phi_1, \dots, \phi_N\}$ (we pick $\beta = 1/\sqrt{2}$).

$\rightsquigarrow$ Now, we want to guarantee that $\sum_{i \in I^c} \epsilon_{i,2}^2 \leq (1 - \beta^2) \epsilon^2 e_2(0)^2$.

$\rightsquigarrow$ Initialize $I = \{1\}$, $I^c = \{2, \dots, N\}$, $\forall i \in I^c, n_i = 1$.

Adaptive algorithm (glimpse !)

Inputs : Target precision ϵ , Lip. constant L , snapshot matrix $S \in \mathbb{R}^{\mathcal{N} \times m}$ ($m < \mathcal{N}$).

$\rightsquigarrow$ Apply SVD on S to compute singular vectors $\phi_1, \phi_2, \dots, \phi_m$.

$\rightsquigarrow$ Find N such that $\|id - P_{X_N}\|_2 \leq \beta \epsilon e_2(0)$ with $X_N = \text{span}\{\phi_1, \dots, \phi_N\}$ (we pick $\beta = 1/\sqrt{2}$).

$\rightsquigarrow$ Now, we want to guarantee that $\sum_{i \in I^c} \epsilon_{i,2}^2 \leq (1 - \beta^2) \epsilon^2 e_2(0)^2$.

$\rightsquigarrow$ Initialize $I = \{1\}$, $I^c = \{2, \dots, N\}$, $\forall i \in I^c, n_i = 1$.

For all $i \in I^c$:

- Compute $\bar{\epsilon}_{i,2} = \omega_i(1 - \beta)^2 \epsilon^2 e_2(0)^2$ and $\bar{\gamma}_i = \omega_i(L^2 - 1)$
- Compute f_i
- While $\epsilon_{i,2} > \bar{\epsilon}_{i,2}$ and $\gamma_i > \bar{\gamma}_i$, $\left(\epsilon_{i,2} := \|a_i - f_i(g_1(a), \dots, g_{n_i}(a))\|_2 \right)$
 - ▶ $n_i \leftarrow n_i + 1$ and recompute f_i (until $n_i = i - 1$)
- If $n_i = i - 1$ and condition still not verified, $I \leftarrow I \cup \{i\}$

$$D(a) = \sum_{i \in I} a_i \phi_i + \sum_{i \in I^c} f_i(g_1(a), \dots, g_{n_i}(a)) \phi_i$$

5 - Numerical illustrations

- 1 Introduction
- 2 Linear approximation
- 3 Nonlinear approximation
- 4 Decoder based on compositional polynomial networks
- 5 Numerical illustrations**

Numerical illustration: KdV

We consider the Korteweg-de Vries (KdV) equation

$$\frac{\partial u}{\partial t} + 4u \frac{\partial u}{\partial x} + \frac{\partial^3 u}{\partial x^3} = 0 \quad \text{on} \quad [-\pi, \pi] \times [0, 1]$$

with periodic boundary conditions and some initial condition.

- $u(\cdot, t) \in \mathbb{R}^{\mathcal{N}}$ ($\mathcal{N} = 512$),
- We train on $t \in [0, 0.2]$ (1000 samples).
- We test on $t \in (0.2, 1]$ (4000 samples).
- $K = \{u(\cdot, t) : t \in [0, 1]\}$.

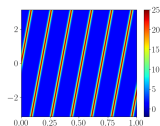


Figure: Function $u(x, t)$.

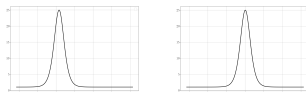


Figure: Some snapshots.

Numerical illustration : KdV

<i>Method</i>	n	N	p	RE_{train}	RE_{test}
Linear	5	/	/	0.435	0.450
Quadratic	5	20	2	0.094	0.099
Neural Networks	5	43	/	0.161	0.166
Sparse	5	43	5	0.017	0.018
CPN-S ($\varepsilon = 10^{-4}$)	5	43	5	0.000072	0.000074

Table: Comparison of methods for the same manifold dimension $n = 5$.

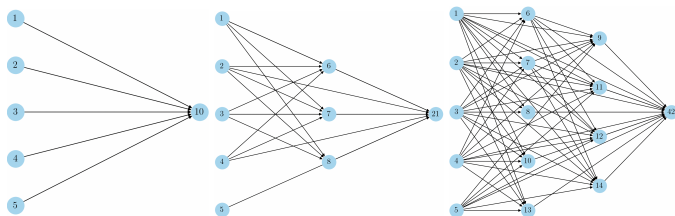


Figure: Some compositional networks for CPN-S ($n = 5, N = 43$).

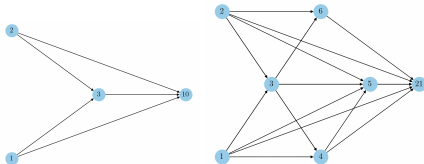
Learning procedure CPN-S

Step	Indices of input coeffs.	Indices of learnt coeffs.
1	1	/
2	1, 2	/
3	1, 2, 3	/
4	1, 2, 3, 4	6
5	1, 2, 3, 4, 5	8, 10, 13
6	1, 2, 3, 4, 5, 6	7, 9, 11, 12, 14, 15, 17
7	1, ..., 5, 6, 7	16, 18, 19, 20
8	1, ..., 5, 6, 7, 8	21, 22, 24, 25, 27, 29
9	1, ..., 5, 6, 7, 8, 9	23, 26, 28
10	1, ..., 5, 6, ..., 10	30
11	1, ..., 5, 6, ..., 11	32, 33, 34, 36, 38
12	1, ..., 5, 6, ..., 12	31, 35, 37
13	1, ..., 5, 6, ..., 13	39, 40, 41
14	1, ..., 5, 6, ..., 14	42
15	1, ..., 5, 6, ..., 15	43

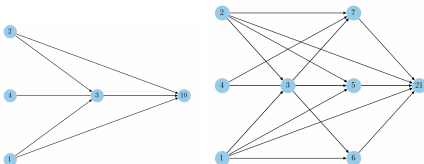
Table: (KdV) Learning procedure of CPN-S for $p = 5$ and $\varepsilon = 10^{-4}$. To reach the target precision, $N = 43$ is required.

Compositional graphs

$$\rightsquigarrow \epsilon = 10^{-2} (n = 2, N = 25)$$



$$\rightsquigarrow \epsilon = 10^{-3} (n = 3, N = 34)$$



Comparison between Sparse and CPN-S

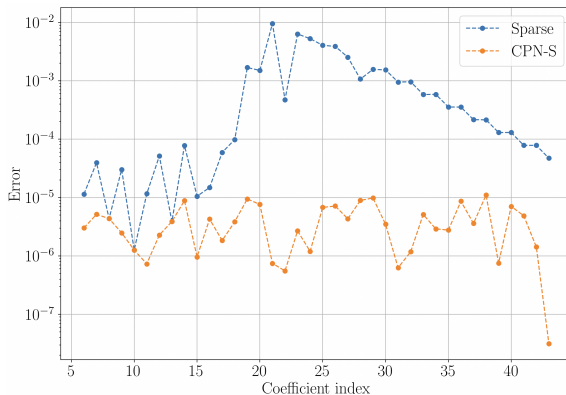


Figure: Coefficient-wise errors for Sparse and CPN-S for $\varepsilon = 10^{-4}$.

Numerical illustration : Inviscid Burgers (Traveling shock problem)

$$\frac{\partial u_\lambda}{\partial t} + \frac{1}{2} \left(\frac{\partial u_\lambda^2}{\partial x} + \frac{\partial u_\lambda v_\lambda}{\partial y} \right) = 0,$$

$$\frac{\partial v_\lambda}{\partial t} + \frac{1}{2} \left(\frac{\partial u_\lambda v_\lambda}{\partial x} + \frac{\partial v_\lambda^2}{\partial y} \right) = 0, (x, y) \in (0, 50), t \in (0, 10).$$

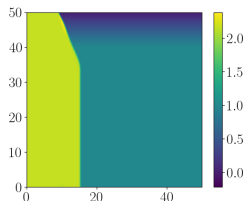
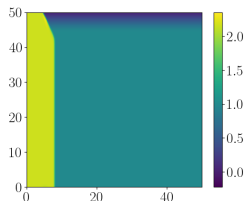
with

ICs : $u_\lambda(x, y, 0) = v_\lambda(0, y, t) = 1$.

BCs : $u_\lambda(0, y, t) = \lambda$. $u_\lambda(x, 0, t) = v_\lambda(0, y, t) = v_\lambda(x, 0, t) = 0$.

$u_\lambda, v_\lambda \in \mathbb{R}^{\mathcal{X} \times \mathcal{Y}}$ ($\mathcal{X} = 250$).

$K = \{(u_\lambda, v_\lambda) : \lambda \in [1.5, 2.5]\}$.



Numerical illustration : Traveling shock problem

<i>Method</i>	n	N	p	RE_{train}	RE_{test}
Linear	9	/	/	0.056	0.050
Quadratic	9	54	2	0.025	0.028
Neural Networks	9	89	/	0.026	0.024
Sparse	9	89	8	0.016	0.016
CPN-S (5.10^{-3})	9	89	8	0.004	0.005

Table: Comparison of methods for the same manifold dimension $n = 9(N = 89)$.

- New model reduction method based on compositional polynomial networks.
- The final decoder is a composition of polynomials, and constructed using an adaptive algorithm.
- The adaptive strategy guarantees a control over the error and the stability of the approximation.

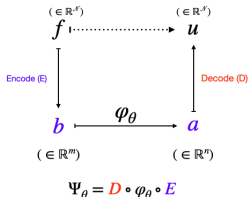
Next steps

$$(\mu, f) \xrightarrow{F} a \xrightleftharpoons[E]{D} \tilde{u}$$

- Operator learning method :

$$\Psi : f \mapsto u$$

We want to find an approximation Ψ_θ of Ψ .



Classical :

$\rightsquigarrow$ PCA on $f : b := (b_i)_{i=1}^m = ((f|\varphi_i^{in}))_{i=1}^n$

$\rightsquigarrow$ PCA on $u : a := (a_i)_{i=1}^n = ((u|\varphi_i^{out}))_{i=1}^n$

$\rightsquigarrow \varphi_\theta : \mathbb{R}^m \rightarrow \mathbb{R}^n$ is trained using data $\{a^{(k)}, b^{(k)}\}_{k=1}^{N_s}$.

Idea :

$\rightsquigarrow$ Replace PCA on u by CPN to reduce the required dimension n .

Online approximation : (For a new parameter μ , $\mu \mapsto u$)

Use Natural Gradient Descent (NGD) to solve the optimization problem

$$a^*(\mu) = \arg \min_{a(\mu) \in \mathbb{R}^n} \|r(D(a(\mu)))\|_2^2$$

where r is the residual of a PDE.

References I



A. Bensalah, A. Nouy and J. Soffo

Nonlinear manifold approximation using Compositional Polynomial Networks.
SIAM, Journal of Scientific Computing...soon.



J. Barnett and C. Farhat.

Quadratic approximation manifold for mitigating the kolmogorov barrier in nonlinear projection-based model order reduction.
Journal of Computational Physics, 464:111348, Sept. 2022.



J. Barnett, C. Farhat, and Y. Maday.

Neural-network-augmented projection-based model order reduction for mitigating the Kolmogorov barrier to reducibility.
J. Comput. Phys., 492:112420, Nov. 2023.



R. Geelen, S. Wright, and K. Willcox.

Operator inference for non-intrusive model reduction with quadratic manifolds.
Computer Methods in Applied Mechanics and Engineering, 403:115717, Jan. 2023.



R. Geelen, L. Balzano, S. Wright, and K. Willcox.

Learning physics-based reduced-order models from data using nonlinear manifolds.
Chaos, 34(3):033122, Mar. 2024.



A. Cohen, C. Farhat, Y. Maday, and A. Somacal.

Nonlinear compressive reduced basis approximation for PDE's.
Comptes Rendus. Mécanique, 351(S1):357–374, 2023.



P. Schwerdtner, B. Peherstorfer.

Greedy construction of quadratic manifolds for nonlinear dimensionality reduction and nonlinear model reduction.

Thank you !

THANK YOU FOR YOUR ATTENTION

Backup slide : Sums of univariate high-order polynomials

$\varphi_1, \dots, \varphi_N$ are orthonormal vectors chosen based on empirical PCA.

$$D(a) = \sum_{i=1}^n a_i \varphi_i + \sum_{i=n+1}^N g_i(a) \varphi_i.$$

$$g_i(a) = \sum_{\ell=2}^p \lambda_{1,\ell} a_1^\ell + \dots + \sum_{\ell=2}^p \lambda_{n,\ell} a_n^\ell.$$

Backup slide : Sums of univariate high-order polynomials

$\varphi_1, \dots, \varphi_N$ are orthonormal vectors chosen based on empirical PCA.

$$D(a) = \sum_{i=1}^n a_i \varphi_i + \sum_{i=n+1}^N g_i(a) \varphi_i.$$

$$g_i(a) = \sum_{\ell=2}^p \lambda_{1,\ell} a_1^\ell + \dots + \sum_{\ell=2}^p \lambda_{n,\ell} a_n^\ell.$$

Alternating minimization algorithm

While *stopping criterion* not satisfied :

- Step 1 : $\min_{(\varphi_i)_{i=1}^N} \sum_{k=1}^m \|u_k - D(a^k)\|_X^2$, such that the $(\varphi_i)_{i=1}^N$ are orthonormal vectors (Orthogonal Procrustes).
- Step 2 : $\min_{(\lambda_{i,\ell})} \sum_{k=1}^m \|u_k - D(a^k)\|_X^2$.
- Step 3 : $\min_{\mathbf{a}^k} \|u_k - D(a^k)\|_X^2, k = 1, \dots, m$.